

AUDITORIA FORENSE DE APLICAÇÃO WEB & NUVEM

Relatório de Auditoria de Segurança

Sistema de Gestão Escolar e Diário Digital (DC-Digital)

DATA DA AUDITORIA

28 de Agosto de 2026

VERSÃO DO SISTEMA

1.0.0 (Build Produção / Vercel + Supabase)

ESCOPO AVALIADO

Código-Fonte Completo (Frontend, Edge Functions, SQL RLS, Criptografia, IndexedDB)

METODOLOGIA

Inspeção Estática Baseada em OWASP Top 10 & 5 Vetores Críticos

Nota Metodológica e Mapeamento da Stack

O sistema **DC-Digital** foi detectado como uma aplicação web moderna com arquitetura *offline-first*. O ecossistema compreende **TypeScript / React 19 / Vite 6** no frontend, **Dexie.js (IndexedDB)** com criptografia Web Crypto (AES-GCM 256 bits) para persistência local, **Supabase (PostgreSQL 17, GoTrue Auth e Edge Functions em Deno)** no backend, e hospedagem no **Vercel** com cabeçalhos de segurança estritos (CSP, HSTS).

As cinco categorias de auditoria foram adaptadas para as camadas reais do ecossistema:

- 1. Banco Sem Tranca:** Avaliação das políticas de Row Level Security (RLS) no PostgreSQL, mapeando filtros de multitenancy por escola (`escola_id`), isolamento de turmas/professores e proteção de dados de alunos.
- 2. Permissão no Navegador:** Confronto entre as restrições de interface (`ProtectedRoute`, flags de papel) e as políticas de validação efetiva nas Edge Functions e no banco.
- 3. IDOR:** Análise de manipuladores nas Edge Functions (`admin-create-user`, `lgpd-request`) e operações de exclusão/atualização parametrizadas por ID.
- 4. Chaves Expostas:** Varredura de histórico Git, fallbacks no bundle JS, variáveis de ambiente (`VITE_*` vs segredos de backend) e configurações locais.
- 5. Inputs sem Tratamento / XSS:** Análise de sinks de renderização (`dangerouslySetInnerHTML`), vetores em exportação de planilhas (CSV Formula Injection) e sanitização em Edge Functions.

React 19

TypeScript

Supabase PostgreSQL

Edge Functions (Deno)

Dexie / IndexedDB

Web Crypto AES-GCM

Vercel CSP

Relatório gerado automaticamente por ferramenta de auditoria de segurança estática. Classificação: **Estritamente Confidencial**.

1. Resumo Executivo

Visão Geral Forense

1
CRÍTICA**2**
ALTA**3**
MÉDIA**4**
BAIXA**7**
PONTOS FORTES

Distribuição de Achados por Severidade



Achados por Categoria Auditada

1. Banco s/ Tranca	3
2. Permissão UI	3
3. IDOR	1
4. Chaves / Defaults	2
5. XSS / Formula Inj.	1

2. Pontos Fortes e Riscos Centrais

✓ Pontos Fortes em Destaque

RLS Granular em Alunos e Notas Pedagógicas

alunos, frequências e notas possuem isolamento rigoroso. Alunos só acessam suas próprias notas e faltas (`aluno_e_o_proprio`), e professores só acessam turmas em que lecionam via `professor_tem_acesso_a_turma()`.

Criptografia AES-GCM no IndexedDB (LGPD)

Campos sensíveis de alunos (`nome`, `cpf`) são cifrados localmente antes de serem persistidos no navegador através de chaves não-exportáveis derivadas via Web Crypto API (`offlineStorage.ts:137-156`).

Defesa em Profundidade na Edge Function

A Edge Function `admin-create-user` valida a role a partir de `app_metadata` assinado pelo JWT (não manipulável no cliente) e bloqueia escalção de privilégios de gestores/secretários para ADMIN.

⚠ Riscos Centrais (Pontos Fracos)

IDOR no Expurgo de Contas de Acesso

A Edge Function `admin-create-user` permite que usuários não-admin com privilégio de exclusão excluam usuários de outras escolas quando o registro não está mapeado na tabela pública `usuarios`.

Exclusão Cruzada de Professores Multi-Escola

Secretários de uma escola possuem permissão RLS para excluir ou atualizar o cadastro global de professores que lecionam em múltiplas escolas da rede municipal.

Desalinhamento de Permissão de Reabertura de Bimestre

A UI exibe botão de "Reabrir aparata" para Gestores e Secretários, mas a política DELETE em `fechamentos_bimestres` bloqueia a operação no banco (apenas ADMIN).

3. Tabela Geral de Achados

Mapeamento Completo

Severidade	Categoria	Arquivo & Linha	Descrição Sintética
CRÍTICA	3. IDOR	supabase/functions/admin-create-user/index.ts:175-203	Bypass de verificação de escola na ação delete-user quando o alvo não consta em usuarios, permitindo exclusão em auth.users de outras escolas.
ALTA	1. Banco Sem Tranca	supabase/migrations/20260707000000_schema.sql:995-1025	Permissão de exclusão/atualização global de professores compartilhados por secretários de uma única escola alocada.
ALTA	1. Banco Sem Tranca	supabase/migrations/20260707000000_schema.sql:530, 536, 1046, 1053	Políticas de escrita de alunos e turmas usam escola_id = get_user_escola_id(), bloqueando escrita sistêmica de GESTOR.
MÉDIA	2. Permissão Navegador	src/pages/AparataDetalhes.tsx:145-146 vs schema.sql:756	UI autoriza Gestores/Secretários a reabrir bimestres (canReabrir), mas RLS em fechamentos_bimestres rejeita exclusão para não-admins.
MÉDIA	2. Permissão Navegador	src/pages/administracao/TabEscolas.tsx:134 vs schema.sql:748	Aba de escolas oculta na UI para não-admins, porém RLS admin_update_escolas permite UPDATE por qualquer GESTOR/SECRETARIO via PostgREST.
MÉDIA	5. Inputs / XSS	src/components/DataExportButton.tsx:18-20	Exportação de dados pessoais e relatórios acadêmicos sem sanitização de caracteres de fórmula de planilhas (=, +, -, @).
BAIXA	1. Banco Sem Tranca	supabase/migrations/20260813000011_fix_rls.sql:276-283	SELECT em professor_horarios restrito por escola_id = get_user_escola_id(), impedindo professores com escola_id nulo de lerem sua grade.
BAIXA	2. Permissão Navegador	src/App.tsx:130-133 vs fix_rls.sql:150-184	Rota /curriculo liberada no React Router para SECRETARIO, mas rejeitada por RLS no PostgreSQL (apenas ADMIN/GESTOR).

Severidade	Categoria	Arquivo & Linha	Descrição Sintética
BAIXA	4. Chaves Expostas	src/lib/supabase.ts:13-14	Dummy JWT anon de fallback hardcoded no bundle público do frontend em vez de bloquear inicialização ou injetar via configuração segura.
BAIXA	5. Inputs / XSS	supabase/functions/lgpd-request/index.ts:122-186	Ingestão e armazenamento de strings brutas em lgpd_requests sem strip/escape de tags HTML na Edge Function.

4. Detalhamento Técnico dos Achados por Categoria

Achado SEC-01 — IDOR na Exclusão de Contas de Acesso

CRÍTICA

ARQUIVO & LINHAS:

supabase/functions/admin-create-user/index.ts:175-218

TRECHO DE CÓDIGO VULNERÁVEL:

```
const { data: targetUserData } = await targetUserQuery.maybeSingle();

// Se for não-ADMIN, verificar se tem permissão para deletar este usuário
if (effectiveRole !== "ADMIN") {
  const { data: callerData } = await supabaseAdmin.from("usuarios").select("escola_id").eq("id", callerUser.id).maybeSingle();
  // VULNERABILIDADE: Se targetUserData for null, o bloco abaixo é ignorado!
  if (targetUserData && (targetUserData.escola_id !== callerData.escola_id || ![ "PROFESSOR", "ALUNO" ].includes(targetUserData.cargo))) {
    return new Response(JSON.stringify({ error: "Você não tem permissão para excluir este usuário" }), { status: 403, ... });
  }
}
// Prossegue e deleta em auth.users via service_role:
if (authUserId) { await supabaseAdmin.auth.admin.deleteUser(authUserId); }
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

Quando um usuário com perfil SECRETARIO ou GESTOR envia uma requisição POST com action: "delete-user" para a Edge Function informando o e-mail de uma conta que existe em auth.users mas não possui registro sincronizado em public.usuarios (ex: conta recém-criada, usuário de suporte ou aluno com e-mail órfão), o check de pertencimento à escola é contornado porque targetUserData é null. A função executa deleteUser() com service_role, permitindo que um secretário exclua contas arbitrárias de outras escolas ou administradores.

Achado SEC-02 — Exclusão Cruzada de Professores Multi-Escola no RLS**ALTA****ARQUIVO & LINHAS:**

supabase/migrations/20260707000000_schema.sql:992-998, 1017-1026

TRECHO DE CÓDIGO VULNERÁVEL:

```
CREATE POLICY "admin_delete_professores" ON public.professores
FOR DELETE
USING ((( SELECT get_user_role() ) = 'ADMIN') OR (is_admin_or_staff() AND (EXISTS ( SELECT 1
FROM professor_alocacoes pa
WHERE ((pa.professor_id = professores.id) AND (pa.escola_id = get_user_escola_id())))))));
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

Na rede municipal, professores lecionam frequentemente em mais de uma unidade escolar (ex: Escola A no turno matutino e Escola B no vespertino). A política RLS permite que o secretário da Escola A delete o registro mestre na tabela professores, pois existe uma alocação para sua escola. Ao deletar o registro mestre, a integridade referencial ou trigger cascadeia a exclusão, apagando o professor e impactando as turmas da Escola B sem o conhecimento daquela gestão.

Achado SEC-03 — Bloqueio de Escrita Sistemática do GESTOR em Alunos e Turmas**ALTA****ARQUIVO & LINHAS:**

supabase/migrations/20260707000000_schema.sql:530, 536, 1046, 1053, 1067

TRECHO DE CÓDIGO VULNERÁVEL:

```
-- Tabela alunos (INSERT/UPDATE/DELETE)
CREATE POLICY "admin_insert_alunos" ON public.alunos
FOR INSERT
WITH CHECK ((( SELECT get_user_role() ) = 'ADMIN') OR (is_admin_or_staff() AND (escola_id = get_user_escola_id
())));
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

O papel GESTOR foi concebido para possuir visão sistêmica da rede municipal de ensino (consistente com a função p_escola_permitida criada na migration 11 para SELECTs). Contudo, nas políticas de INSERT/UPDATE/DELETE de alunos, turmas, professor_alocacoes e professor_horarios, a condição exige escola_id = get_user_escola_id(). Como o gestor municipal possui escola_id nulo ou fixo em uma escola sede, ele é bloqueado pelo banco ao criar ou remanejar turmas e alunos em outras escolas.

Achado SEC-04 — Desalinhamento na Reabertura de Bimestre (UI vs RLS)**MÉDIA****ARQUIVO & LINHAS:**

src/pages/AparataDetalhes.tsx:145-146 vs supabase/migrations/20260707000000_schema.sql:756-760

TRECHO DE CÓDIGO VULNERÁVEL:

```
// Frontend: AparataDetalhes.tsx
const canReabrir = _user?.role === 'GESTOR' || _user?.role === 'SECRETARIO' || _user?.role === 'ADMIN';

// Backend RLS: schema.sql
CREATE POLICY "admin_delete_fechamentos" ON public.fechamentos_bimestres
  FOR DELETE
  USING ((get_user_role() = 'ADMIN'::text));
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

O frontend expõe o fluxo de reabertura de bimestre para diretores e secretários escolares, permitindo que corrijam pendências de fechamento. Porém, ao executar a ação, o frontend realiza um DELETE na tabela fechamentos_bimestres. Como o RLS só permite DELETE para ADMIN, a requisição é rejeitada pelo PostgreSQL com erro 42501 (insufficient privilege), quebrando a operação de forma silenciosa ou ruidosa no cliente.

Achado SEC-05 — Risco de Injeção de Fórmulas em Exportação de Dados (CSV/JSON)**MÉDIA****ARQUIVO & LINHAS:**

src/components/DataExportButton.tsx:18-20 e páginas de relatórios

TRECHO DE CÓDIGO VULNERÁVEL:

```
const jsonString = `data:text/json;charset=utf-8,{${encodeURIComponent(
  JSON.stringify(data, null, 2)
)}}`;
const downloadAnchor = document.createElement('a');
downloadAnchor.setAttribute('href', jsonString);
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

Se um aluno, responsável ou professor possuir campos cadastrais (nome, endereço, observação de diário, mensagem LGPD) que iniciem com caracteres executáveis de planilhas como =, +, -, @ ou (ex: =cmd|' /C calc '!A0), a exportação para CSV ou conversão de JSON em planilha no Excel/LibreOffice pode acionar execução remota de código (DDE) ou exfiltração de dados locais na máquina do operador da secretaria.

Achado SEC-06 — Política RLS de Escolas Permite UPDATE por Secretário**MÉDIA****ARQUIVO & LINHAS:**

src/pages/administracao/TabEscolas.tsx:134 vs schema.sql:748-753

TRECHO DE CÓDIGO VULNERÁVEL:

```
CREATE POLICY "admin_update_escolas" ON public.escolas
FOR UPDATE
USING ((( SELECT get_user_role() ) = 'ADMIN') OR (is_admin_or_staff() AND (id = get_user_escola_id()))))
```

MECÂNICA DE EXPLORAÇÃO & IMPACTO:

Na interface, a aba de Gestão de Escolas é restrita a administradores. Entretanto, a política RLS no banco permite que qualquer SECRETARIO execute UPDATE nos dados da sua escola via API PostgREST direta, podendo alterar status da escola para "Inativa", modificar código INEP ou URL de logotipos sem autorização da administração municipal.

5. Recomendações Priorizadas de Correção

P1 — Crítica**Blindagem de IDOR na Edge Function admin-create-user**

Garantir que a exclusão de usuários (action: "delete-user") rejeite a operação imediatamente com HTTP 404/403 se o usuário alvo não for localizado e validado como pertencente à mesma escola_id do chamador não-admin.

P2 — Alta**Isolamento de Exclusão de Professores Multi-Escola**

Substituir a exclusão direta na tabela mestre professores por desvinculação em professor_alocacoes. Apenas o papel ADMIN deve ter autorização para deletar o registro global de um professor.

P2 — Alta**Alinhamento de Escrita Sistemática do GESTOR (RLS)**

Atualizar as políticas de INSERT/UPDATE/DELETE em alunos, turmas e professor_alocacoes para utilizarem a função p_escola_permitida(escola_id), habilitando escrita do GESTOR em todas as unidades da rede.

P3 — Média

Alinhamento de Permissões de Reabertura de Bimestre e Gestão de Escolas

Criar política de DELETE/UPDATE em fechamentos_bimestres permitindo que secretários e gestores reabram bimestres de suas escolas vinculadas, e restringir admin_update_escolas a administradores (ou campos seguros não-críticos).

P4 — Baixa

Sanitização Anti-Formula Injection & Hardening de Inputs

Prefixar campos alfanuméricos com aspas simples caso iniciem com =, +, -, @ em exportações de relatórios e arquivos de dados pessoais (LGPD).

6. Issues Prontas para o GitHub

Copie e Cole no Repositório

--- ISSUE 1 ---

```
## [Segurança] IDOR na Edge Function admin-create-user permite exclusão cruzada de contas

**Labels sugeridas:** `security`, `severity:critical`, `backend`

### Descrição do Problema
Na Edge Function `admin-create-user/index.ts`, ao processar a ação `delete-user`, a validação de escola do chamador não-admin verifica apenas se `targetUserData` existe antes de checar `targetUserData.escola_id !== callerData.escola_id`.
Se o usuário alvo existir em `auth.users` mas não possuir registro na tabela pública `usuarios`, `targetUserData` é nulo, fazendo com que o check de segurança seja ignorado e a conta seja deletada em `auth.users` via `supabaseAdmin.auth.admin.deleteUser()`.

### Evidência
- **Arquivo:** `supabase/functions/admin-create-user/index.ts` (Linhas 175-203)
  ```typescript
 const { data: targetUserData } = await targetUserQuery.maybeSingle();
 if (effectiveRole !== "ADMIN") {
 // Se targetUserData for null, o if não entra e a requisição prossegue!
 if (targetUserData && (targetUserData.escola_id !== callerData.escola_id || !["PROFESSOR",
 "ALUNO"].includes(targetUserData.cargo))) {
 return new Response(JSON.stringify({ error: "Você não tem permissão para excluir este usuário" })), {
 status: 403, ... });
 }
 }
 ...
  ```

### Impacto
Um usuário com papel `SECRETARIO` ou `GESTOR` pode revogar e excluir credenciais de acesso de qualquer usuário do sistema (incluindo Administradores e servidores de outras escolas) fornecendo apenas o e-mail alvo.

### Sugestão de Correção
1. Exigir que `targetUserData` seja obrigatoriamente não-nulo para chamadores não-admin.
2. Se não localizado na tabela pública `usuarios` vinculada à escola do chamador, rejeitar a requisição com `404 Not Found` ou `403 Forbidden`.

### Critérios de Aceite
- [ ] Requisições `delete-user` por não-admin para e-mails não cadastrados na tabela `usuarios` da respectiva escola são rejeitadas com 403/404.
- [ ] Não-admin só consegue excluir usuários com cargo PROFESSOR ou ALUNO vinculados estritamente ao seu `escola_id`.
- [ ] Teste automatizado cobrindo tentativa de exclusão de usuário órfão ou de outra escola por Secretário.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Exclusão e alteração cruzada de professores multi-escola via RLS

****Labels sugeridas:**** `security`, `severity:high`, `database`

Descrição do Problema

A política RLS `admin\_delete\_professores` e `admin\_update\_professores` permite que qualquer secretário de uma escola onde o professor possua alocação realize a exclusão ou alteração do registro global na tabela `professores`. Se o professor lecionar em mais de uma escola da rede municipal, a exclusão por uma secretaria remove o professor de todo o sistema.

Evidência

- ****Arquivo:**** `supabase/migrations/20260707000000\_schema.sql` (Linhas 992-1026)

``sql

```
CREATE POLICY "admin_delete_professores" ON public.professores
  FOR DELETE
  USING ((( SELECT get_user_role() ) = 'ADMIN') OR (is_admin_or_staff() AND ( EXISTS ( SELECT 1
    FROM professor_alocacoes pa
    WHERE ((pa.professor_id = professores.id) AND (pa.escola_id = get_user_escola_id())))))));
``
```

Impacto

Quebra de integridade e negação de serviço para outras escolas da rede municipal que compartilham o mesmo professor.

Sugestão de Correção

1. Restringir a exclusão na tabela `professores` estritamente ao papel `ADMIN`.
2. Secretários devem apenas ter permissão para remover registros da tabela `professor\_alocacoes` e `professor\_horarios` vinculados à sua própria escola.

Critérios de Aceite

- [] Secretário da Escola A não consegue deletar registros em `public.professores`.
- [] Secretário da Escola A consegue remover apenas a linha em `professor\_alocacoes` com seu `escola\_id`.
- [] Apenas `ADMIN` consegue executar DELETE na tabela `professores`.

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Alinhamento de permissões de escrita do papel GESTOR e reabertura de bimestres

Labels sugeridas: `security`, `severity:medium`, `database`

Descrição do Problema

1. As políticas de INSERT/UPDATE/DELETE em `alunos`, `turmas` e `professor\_alocacoes` utilizam `escola\_id = get\_user\_escola\_id()`, bloqueando a escrita de usuários com papel `GESTOR` (que possuem escopo municipal).
2. A política RLS `admin\_delete\_fechamentos` em `fechamentos\_bimestres` permite exclusão apenas para `ADMIN`, enquanto o frontend expõe a reabertura de bimestre para `GESTOR` e `SECRETARIO`.

Evidência

- **Arquivo:** `supabase/migrations/20260707000000\_schema.sql` (Linhas 530-552, 756-760, 1046-1069)
- **Arquivo:** `src/pages/AparataDetalhes.tsx` (Linhas 145-146)

Impacto

Bloqueio de operações legítimas da gestão escolar no banco de dados e divergência entre as regras de negócio da UI e as políticas do PostgreSQL.

Sugestão de Correção

1. Atualizar políticas de escrita para utilizar `p\_escola\_permitida(escola\_id)`.
2. Criar política granular em `fechamentos\_bimestres` permitindo que Secretários e Gestores excluam/reabram fechamentos de turmas da sua escola.

Critérios de Aceite

- [] GESTOR municipal consegue criar/editar alunos e turmas em qualquer escola da rede.
- [] SECRETARIO consegue reabrir bimestres (`DELETE` em `fechamentos\_bimestres`) de turmas vinculadas à sua escola.
- [] PROFESSOR e ALUNO continuam impedidos de reabrir bimestres.

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Segurança] Proteção contra Formula Injection (CSV/Excel) em exportações e LGPD

****Labels sugeridas:**** `security`, `severity:medium`, `frontend`

Descrição do Problema

Nas rotas de exportação de dados do Centro de Privacidade (`DataExportButton.tsx`) e relatórios acadêmicos (`RelatorioNotas.tsx`, `RelatorioMedias.tsx`), campos de texto livre controlados pelo usuário (nomes, mensagens LGPD, observações) são exportados diretamente sem sanitização de caracteres de controle de fórmulas do Excel/Calc.

Evidência

- ****Arquivo:**** `src/components/DataExportButton.tsx` (Linhas 18-20)
- ****Arquivo:**** `src/pages/MinhaPrivacidade.tsx` (Linhas 178-193)

Impacto

Execução indevida de comandos DDE ou exfiltração de dados quando relatórios e arquivos JSON/CSV são abertos por operadores no Microsoft Excel ou LibreOffice Calc.

Sugestão de Correção

Implementar função de sanitização de fórmulas que prefixe qualquer string iniciada por `=`, `+`, `-`, `@`, `\t`, `\r` com apóstrofo (`'`) antes de gerar o payload de exportação.

Critérios de Aceite

- [] Strings iniciadas por `=`, `+`, `-`, `@` são sanitizadas com prefixo de escape.
- [] Exportação de dados pessoais e relatórios acadêmicos testada com payloads de fórmula.

--- FIM ISSUE 4 ---